

Model Evaluation and Hyperparameter Tuning

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV,
RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report

# 1. Load and split the data
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 2. Train baseline models
models = {
    'RandomForest': RandomForestClassifier(random_state=42),
    'SVM': SVC(random_state=42)
}
```

```

print(" ♦ Baseline Model Performance:")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"\n{name}:")
    print(f" Accuracy : {accuracy_score(y_test, y_pred):.3f}")
    print(f" Precision: {precision_score(y_test, y_pred, average='macro'):.3f}")
    print(f" Recall  : {recall_score(y_test, y_pred, average='macro'):.3f}")
    print(f" F1-score : {f1_score(y_test, y_pred, average='macro'):.3f}")

```

3. Hyperparameter tuning with GridSearchCV

```

param_grid_rf = {
    'n_estimators': [10, 50, 100],
    'max_depth': [None, 5, 10]
}

grid_rf = GridSearchCV(RandomForestClassifier(random_state=42),
    param_grid_rf, cv=5, scoring='f1_macro')

grid_rf.fit(X_train, y_train)

best_rf = grid_rf.best_estimator_

```

4. Hyperparameter tuning with RandomizedSearchCV

```

param_dist_svm = {
    'C': np.logspace(-2, 2, 5),
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

```

```
rand_svm = RandomizedSearchCV(SVC(random_state=42), param_dist_svm,  
n_iter=10, cv=5, scoring='f1_macro', random_state=42)
```

```
rand_svm.fit(X_train, y_train)
```

```
best_svm = rand_svm.best_estimator_
```

5. Evaluate tuned models

```
print("\n 🎯 Tuned Model Performance:")
```

```
for name, model in [('GridSearch RF', best_rf), ('RandomSearch SVM',  
best_svm)]:
```

```
    y_pred = model.predict(X_test)
```

```
    print(f"\n{name}:")
```

```
    print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

6. Compare and choose best model

```
scores = {
```

```
    'GridSearch RF': f1_score(y_test, best_rf.predict(X_test), average='macro'),
```

```
    'RandomSearch SVM': f1_score(y_test, best_svm.predict(X_test),  
average='macro')
```

```
}
```

```
best_model_name = max(scores, key=scores.get)
```

```
print(f"\n 🏆 Best model: {best_model_name} with F1-score =  
{scores[best_model_name]:.3f}")
```

Output:-

◆ Baseline Model Performance:



RandomForest:

Accuracy : 1.000
Precision: 1.000
Recall : 1.000
F1-score : 1.000

SVM:

Accuracy : 1.000
Precision: 1.000
Recall : 1.000
F1-score : 1.000



Tuned Model Performance:

GridSearch RF:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

RandomSearch SVM:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



Best model: GridSearch RF with F1-score = 1.000